

Selenium

Mature WebDriver ecosystem for browser automation with Java, Python, C#, JavaScript and other languages.

What it is and when to use it

Selenium automates browsers through the WebDriver standard and remains strong in enterprise and multi-language ecosystems.

Prerequisites

Choose the project language and install its runtime/JDK/SDK. For Java configure JDK plus Maven/Gradle; for Python use Python and a virtual environment.

Installation and drivers

Add Selenium through the language package manager. Selenium Manager can resolve drivers in many modern setups; verify browser/driver compatibility when startup fails.

First project

Start WebDriver, navigate, locate elements, perform actions, assert results and close the driver safely.

Locators and waits

Prefer stable IDs/test attributes and explicit waits. Avoid fixed sleeps as synchronization strategy.

Page Object and architecture

Separate page behavior, data, configuration and assertions. Use Page Objects to model behavior, not only selectors.

Grid and parallel execution

Selenium Grid distributes sessions across browsers and machines; size parallelism to data isolation and environment capacity.

CI/CD and evidence

Run headless when appropriate and publish reports, screenshots and browser/driver logs.

Troubleshooting

Common issues include driver mismatch, stale elements, synchronization, frames, windows and shadow DOM.

QA tip

Use the tool within a quality strategy: automate meaningful risks, keep the suite observable and treat automation failures as production-code issues.

Official resources

Site oficial: <https://www.selenium.dev/>

Download / instalação: <https://www.selenium.dev/downloads/>

Documentação: <https://www.selenium.dev/documentation/>